SOFTWARE ARCHITECTURE DESIGN · 2025/26

Music Playlist Manager

Sprint 2 Review · Riproduzione, UI real-time, Undo, Metriche e Retrospettiva

Gruppo 02

Apicella Francesco
Campagna Lorenzo
Cutolo Raffaele
Gioia Errico

11 giugno 2026

Terza consegna

SPRINT 2 · OBIETTIVO

Riproduzione e UI in tempo reale

Obiettivo: a fine Sprint l'utente può avviare, mettere in pausa e saltare la riproduzione, scegliere la modalità a runtime (sequenziale · shuffle · loop), modificare le playlist mentre suonano e annullare l'ultima operazione. Costruisce sopra Model e Service definiti nello Sprint 1.

8
User Stories selezionate

24
Story Points pianificati

~50h
ore-uomo pianificate

36h
capacity (9h × 4 membri)

User Stories dello Sprint 2

US-07a/b	Avvia · pausa · skip
US-08	Scelta modalità (Strategy)
US-09a/b	Aggiungi/rimuovi tracce live
US-10/11	Now Playing + evidenziazione
US-12	Undo aggiunta / rimozione

SPRINT RELEASE · DEMO

Cosa è stato rilasciato

Una riproduzione simulata pienamente funzionante: tutte le 8 US dello Sprint sono "Done".
Di seguito i blocchi che mostreremo nella demo dal vivo.

Controlli di riproduzione

Play, pausa e skip su tracce e playlist. La simulazione gira su una Timeline JavaFX; lo stato (PLAYING/PAUSED/STOPPED) e il tempo trascorso sono conservati alla pausa.

Modalità intercambiabili

Sequenziale, shuffle e loop selezionabili a runtime tramite il pattern Strategy + Factory, senza interrompere la traccia corrente.

UI in tempo reale

Pannello "Now Playing" e playlist con traccia corrente evidenziata, aggiornati via Observer sulle property osservabili del PlaybackService.

Modifica live & Undo

Aggiunta e rimozione di tracce durante la riproduzione senza interruzioni; undo di ogni operazione di aggiunta/rimozione tramite la CommandHistory.

Story Points, velocity e problemi

24
SP pianificati

24
SP completati

24
Velocity (Sprint 2)

8 / 8
User Stories "Done"

Velocity raddoppiata rispetto allo Sprint 1 (12 SP): entrambe le US-buffer (US-11, US-12) completate, nessun rinvio.

PROBLEMI E NOTE EMERSE

Avvio da libreria. play() accettava solo una Playlist: il caso è stato convogliato su un unico percorso (play(Track) incapsula una playlist mono-traccia, principio DRY).

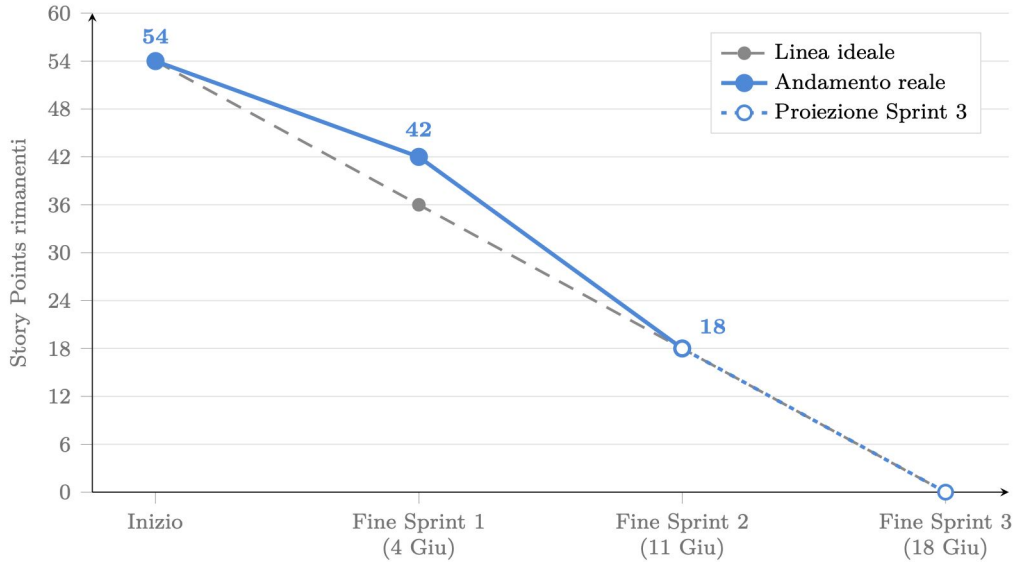
Diagramma classi. Il package util / ValidationUtils, esistente dallo Sprint 1, mancava dal diagramma: documento di architettura aggiornato. Tema ripreso nella Retrospettiva.

Stime. US-09a si è ridotta alla verifica: l'architettura la supportava già. Persistenza (US-18) rinviata allo Sprint 3 con debito di serializzazione da saldare.

BURNDOWN DI PROGETTO

Story Points residui sull'intero backlog

Backlog totale 54 SP. A fine Sprint 2 restano 18 SP: lo scostamento di +6 SP dello Sprint 1 è stato pienamente riassorbito, esattamente sulla linea ideale. La tratteggiata proietta i 18 SP residui sullo Sprint 3.



SPRINT RETROSPECTIVE · STARFISH

Cosa cambiamo nel processo

Keep doing	Integrazione frequente sul branch principale (impegno preso nello Sprint 1, ora abitudine) · design collettivo iniziale prima dello split · code review incrociate con accoppiamento ruotato e ownership per US su tutti i layer.
More of	Predisporre l'architettura in anticipo per le funzionalità future: ha reso US-09a e US-12 a basso costo (infrastruttura Command già pronta dallo Sprint 1).
Start	Allineare periodicamente diagramma classi/package e codice · aggiornare il documento di architettura contestualmente alle modifiche strutturali, non a posteriori.
Less of	Rilevare con ritardo i disallineamenti tra documento di architettura e codice (il PlaylistController nello Sprint 1, il package util in questo sprint).
Stop	Lasciare che documento di architettura e codice divergano senza un momento di riconciliazione pianificato a ogni fine sprint.

PROSSIMI PASSI

Sprint 3 — Funzionalità e persistenza

Con il nucleo funzionale completato (36 SP su 54), lo Sprint 3 chiude l'intero scope: arricchimento del prodotto e persistenza su disco. 18 SP residui = velocity media dimostrata.

Discovery

`US-13` · `US-14a/b` · `US-16`

Home page con le tracce e playlist più riprodotte; generazione automatica di playlist per genere, anno o tag.

Tag & riordino

`US-15` · `US-17`

Tag visivi (preferito, esplicito, nuova uscita) sulle tracce; riordino delle tracce in playlist, anche durante la riproduzione.

Persistenza

`US-18a` · `US-18b`

Salvataggio su disco alla chiusura e caricamento automatico all'avvio (pattern DAO + JSON), saldando il debito di serializzazione.

Grazie per l'attenzione.

github.com/fra343/Gruppo2-SAD
trello.com/b/Mpb9bjYE/sad